

```

17. def load_resnet50_model():
18.     """
19.     Load the pre-trained ResNet50 model.
20.     """
21.     model = models.resnet50(pretrained=True)
22.     model.eval()
23.     return model

```

We are loading the pretrained ImageNet weights in both cases. **Note** that we need the ResNet50 model only for the comparison part. For standalone image classification, we will just use the EfficientNetB0 model.

The next function preprocesses the image before being fed into the neural network model. We also have another function to read the ImageNet classes from `imagenet_classes.txt` file.

utils.py

```

24. def preprocess():
25.     """
26.     Define the transform for the input image/frames.
27.     Resize, crop, convert to tensor, and apply ImageNet normalization stats.
28.     """
29.     transform = transforms.Compose([
30.         transforms.ToPILImage(),
31.         transforms.Resize(256),
32.         transforms.CenterCrop(224),
33.         transforms.ToTensor(),
34.         transforms.Normalize(mean=[0.485, 0.456, 0.406],
35.                               std=[0.229, 0.224, 0.225]),])
36.     return transform
37.
38. def read_classes():
39.     """
40.     Load the ImageNet class names.
41.     """
42.     with open("imagenet_classes.txt", "r") as f:
43.         categories = [s.strip() for s in f.readlines()]
44.     return categories

```

We are cropping the image, resizing it to 224×224, converting it to tensor, and applying the ImageNet normalization.

The final two functions are to carry out forward pass through the model multiple times and to plot the time vs iterations graph.

utils.py

```

45. def run_through_model(model, input_batch, n_runs):
46.     """
47.     Forward passes the image tensor batch through the model
48.     `n_runs` number of times. Prints the average milliseconds taken
49.     and returns a list containing all the forward pass times.
50.     """
51.     total_time = 0
52.     time_list = []
53.     for i in range(n_runs):
54.         print(f"Run: {i+1}", end='\r')
55.         with torch.no_grad():
56.             start_time = time.time()
57.             output = model(input_batch)
58.             end_time = time.time()
59.             time_list.append((end_time-start_time)*1000)
60.             total_time += (end_time - start_time)*1000
61.
62.     print(f"{total_time/n_runs:.3f} milliseconds\n")
63.     return time_list
64.
65. def plot_time_vs_iter(model_names, time_lists, device):
66.     """
67.     Plots the iteration vs time graph for given model.
68.
69.     :param model_name: List of strings, name of both the models.
70.     :param time_list: List of lists, containing time take for each iteration
71.                       for each model.
72.     :param device: Computation device.
73.     """

```